

11 PySpark SQL - NikhilSharma

PySpark SQL

1. Introduction to Spark SQL

Spark SQL is a powerful module within Apache Spark that bridges the gap between relational database management systems and distributed data processing. It allows users to interact with structured and semi-structured data using standard SQL queries, while leveraging Spark's robust execution engine for large-scale data manipulation.

1.1 Why Do We Need Spark SQL?

- SQL Familiarity:** Many data professionals, analysts, and engineers are proficient in SQL. Spark SQL allows them to leverage their existing SQL knowledge to work with big data in Spark, significantly reducing the learning curve compared to purely programmatic APIs.
- Unified Data Access:** It provides a single interface to query data from various sources (e.g., Parquet, ORC, JSON, CSV, JDBC, Hive) and formats, treating them all as tables.
- Optimized Performance:** At its core, Spark SQL utilizes the **Catalyst Optimizer**, a sophisticated query optimizer that analyzes, plans, and optimizes SQL queries and DataFrame operations. This ensures that queries are executed efficiently, often outperforming hand-tuned code for complex operations.
- Integration with Spark Ecosystem:** Spark SQL seamlessly integrates with other Spark components like Spark Streaming, MLlib (Machine Learning), and GraphX, enabling end-to-end data pipelines.
- Declarative vs. Programmatic:** While DataFrame API offers a programmatic way to manipulate data, Spark SQL provides a declarative approach. You specify *what* you want to achieve (e.g., `SELECT`, `GROUP BY`), and Spark figures out *how* to execute it efficiently.

1.2 Spark SQL vs. DataFrame API: Key Differences

Both Spark SQL and the DataFrame API are fundamental for working with structured data in PySpark, and they are deeply intertwined. In fact, Spark SQL queries are internally converted into DataFrame operations, and vice versa.

Feature	Spark SQL	DataFrame API
Nature	Declarative (SQL strings)	Programmatic (method chaining)
Syntax	Standard SQL syntax	Python, Scala, Java, R specific methods
Readability	Often more readable for complex joins/logic	Can be more verbose for complex operations
Flexibility	Less flexible for custom logic	More flexible for custom UDFs, complex transformations
Error Debug	Errors might be harder to pinpoint in SQL	Easier to debug with stack traces
Use Case	Ad-hoc queries, BI tools, SQL-savvy teams	ETL pipelines, data science, programmatic control

Interchangeability:

- You can register a DataFrame as a temporary view and then query it using Spark SQL.
- The result of a Spark SQL query is always a DataFrame, which can then be further manipulated using the DataFrame API.

2. Key Concepts in Spark SQL

2.1 SparkSession: The Entry Point

`SparkSession` is the unified entry point for all Spark functionalities, including Spark SQL, DataFrames, and Dataset API. It replaces the older `SQLContext` and `HiveContext`.

Example: Creating a SparkSession

```
from pyspark.sql import SparkSession

# Initialize SparkSession
# .builder: Provides a builder for SparkSession
```

```
# .appName("Spark SQL Example"): Sets a name for your Spark application
# .getOrCreate(): Returns an existing SparkSession or creates a new one if it doesn't exist
spark = SparkSession.builder \
    .appName("Spark SQL Example") \
    .getOrCreate()
```

2.2 DataFrames as Tables/Views

To execute SQL queries on a DataFrame, you must register it as a table or view within the SparkSession's catalog.

- **DataFrame:** A distributed collection of data organized into named columns. It's conceptually equivalent to a table in a relational database or a data frame in R/Python.
- **View:** A virtual table based on the result-set of a SQL query. It does not store data itself but rather defines how to retrieve data from its underlying DataFrame.

```
download_url =
"https://raw.githubusercontent.com/KirkYagami/PySpark_Training/refs/heads/main/PySpark/03_data/disney_plus_shows.csv"

import requests
response = requests.get(download_url)

if response.status_code == 200:
    with open('disney_plus_shows.csv', 'wb') as f:
        f.write(response.content)
else:
    print("Failed to download the file")

# Load data into DataFrame
disney_df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("multiline", "true") \
    .option("quote", "\"") \
    .option("escape", "\\") \
    .load("disney_plus_shows.csv")
```

createOrReplaceTempView()

- **Scope:** The view created using `createOrReplaceTempView()` is **session-scoped**. This means it is only accessible within the `SparkSession` in which it was created.
- **Lifetime:** It is **ephemeral** (temporary). The view is dropped automatically when the `SparkSession` terminates.
- **Purpose:** Ideal for ad-hoc queries or intermediate steps within a single application session.

Example: Registering a DataFrame as a Temporary View

```
disney_df.createOrReplaceTempView("disney_shows")
```

createOrReplaceGlobalTempView()

- **Scope:** The view created using `createOrReplaceGlobalTempView()` is **cluster-scoped**. It is accessible across all `SparkSession`s running on the same Spark application (cluster).
- **Lifetime:** It is also **ephemeral**, meaning it's dropped when the Spark application (e.g., `spark-submit job`) terminates.
- **Access:** To query a global temporary view, you must prefix its name with `global_temp`. For example, `SELECT * FROM global_temp.my_global_view`.
- **Purpose:** Useful when you need to share a view across multiple isolated `SparkSession`s within the same Spark application.

Example: Registering a DataFrame as a Global Temporary View

```
# Assuming 'disney_df' is already loaded
disney_df.createOrReplaceGlobalTempView("disney_shows_global")

# To query it later:
# result_global = spark.sql("SELECT * FROM global_temp.disney_shows_global")
```

Why Create Views?

1. **SQL Interface:** Allows you to use familiar SQL syntax to query your DataFrames.
2. **Abstraction:** Hides the underlying data source and complex DataFrame operations, presenting a simplified table-like interface.
3. **Reusability:** Once a view is created, it can be queried multiple times by different SQL statements without redefining the underlying DataFrame logic.
4. **Optimization:** Spark's Catalyst Optimizer works on these views, applying optimizations to the SQL queries executed against them, potentially leading to better performance than manual DataFrame API chaining for complex scenarios.

2.3 Schema Association

- **DataFrame Schema:** When you load data into a DataFrame, Spark infers or you explicitly define a schema (structure of the data, including column names and data types). This schema is inherent to the DataFrame.
- **View Schema:** When a DataFrame is registered as a temporary or global temporary view, its schema is automatically associated with that view in the Spark Catalog. This allows Spark SQL to understand the structure of the data when you write queries against the view.
- **Persistence:** For persistent tables (e.g., Hive tables, managed tables in Delta Lake), the schema is stored in an external metastore (like Hive Metastore or Delta Lake transaction log), making it available across Spark applications and sessions.

2.4 Spark Catalog (Runtime Catalog)

The Spark Catalog, accessible via `spark.catalog`, is Spark SQL's **runtime metadata repository**. It acts as a central registry for all relational entities (databases, tables, views, functions) that Spark knows about within the current `SparkSession` or across the Spark application (for global views).

What it stores:

- Information about temporary views and global temporary views.
- References to external tables (e.g., Hive tables, Delta tables).
- Registered user-defined functions (UDFs).
- Database names.

How it's used:

- **Name Resolution:** When you write a SQL query like `SELECT * FROM my_table`, Spark uses the Catalog to resolve `my_table` to its corresponding DataFrame or underlying data source.
- **Metadata Management:** It allows you to programmatically inspect the available tables, columns, and databases.
- **Session Isolation:** For temporary views, the Catalog ensures that views are isolated to their respective `SparkSessions`.

Example: Interacting with the Catalog

```
# List all tables/views in the current session
print("Tables in current session:")
spark.catalog.listTables().show()

# List all columns for a specific table/view
print("\nColumns for 'disney_shows':")
spark.catalog.listColumns("disney_shows").show()

# You can also list databases
```

```
print("\nDatabases:")
spark.catalog.listDatabases().show()
```

2.5 Catalyst Optimizer

While not directly a concept you "use" in your SQL queries, understanding the Catalyst Optimizer is crucial for appreciating Spark SQL's performance.

- **Role:** Catalyst is Spark SQL's query optimizer. It transforms your SQL queries and DataFrame operations into an optimized physical execution plan.
- **Phases:**
 1. **Analysis:** Resolves references (e.g., column names, table names) against the Catalog.
 2. **Logical Optimization:** Applies rule-based optimizations (e.g., predicate pushdown, column pruning, join reordering).
 3. **Physical Planning:** Generates multiple physical plans and selects the most efficient one based on cost (e.g., choosing between sort-merge join or broadcast join).
 4. **Code Generation:** Generates optimized Java bytecode for the execution plan.
- **Benefit:** This optimization process happens automatically, allowing users to write high-level SQL or DataFrame code without worrying about low-level performance tuning.

3. Setting Up Spark SQL

Before using Spark SQL, you need to initialize a `SparkSession`, which is the entry point for Spark SQL operations.

Example: Creating a SparkSession

```
from pyspark.sql import SparkSession

# Initialize SparkSession
spark = SparkSession.builder \
    .appName("Spark SQL Example") \
    .getOrCreate()
```

4. Loading Data with Spark SQL

You can load data into Spark SQL DataFrames from various sources. For this example, we'll use the `disney_plus_shows.csv` dataset.

Example: Loading CSV Data

```
# Load CSV data into DataFrame
disney_df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("multiline", "true") \
    .option("quote", "\"") \
    .option("escape", "\\") \
    .load("disney_plus_shows.csv")

# Show schema and first few rows
disney_df.printSchema()
disney_df.show(truncate=False)
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, FloatType, \
    DateType
from pyspark.sql.functions import col, regexp_replace, to_date

cleaned_disney_df = disney_df \
    .withColumn("released_date", to_date(col("released_at"), "dd MMM yyyy")) \
    .withColumn("added_date", to_date(col("added_at"), "MMMM dd, yyyy")) \
```

```

    .withColumn("runtime_minutes", regexp_replace(col("runtime"), " min",
    "").cast(IntegerType())) \
    .withColumn("metascore_int", col("metascore").cast(IntegerType())) \
    .withColumn("imdb_rating_float", col("imdb_rating").cast(FloatType())) \
    .withColumn("imdb_votes_int", regexp_replace(col("imdb_votes"), ",", "").cast(IntegerType())) \
    .drop("released_at", "added_at", "runtime", "metascore", "imdb_rating", "imdb_votes")

print("\nSchema after cleaning and casting with DataFrame API:")
cleaned_disney_df.printSchema()
cleaned_disney_df.show(5, truncate=False)

```

5. Registering DataFrames as SQL Tables/Views

Once the DataFrame is loaded, you can register it as a temporary SQL table to run SQL queries.

Example: Registering a DataFrame

```

# Register DataFrame as a SQL table (session-scoped, temporary)
disney_df.createOrReplaceTempView("disney_shows")

# Optionally, register as a global temporary view (cluster-scoped, temporary)
disney_df.createOrReplaceGlobalTempView("disney_shows_global")

```

6. Executing SQL Queries

With the DataFrame registered as a SQL table, you can now run SQL queries on it using `spark.sql()`.

```

spark.sql("""
SELECT
    title,
    CAST(year AS INT) AS year_int,
    TO_DATE(released_at, 'dd MMM yyyy') AS released_date_sql,
    CAST(REPLACE(runtime, ' min', '') AS INT) AS runtime_minutes_sql,
    CAST(metascore AS INT) AS metascore_int_sql,
    CAST(imdb_rating AS FLOAT) AS imdb_rating_float_sql,
    CAST(REPLACE(imdb_votes, ',', '') AS INT) AS imdb_votes_int_sql,
    genre
FROM
    disney_shows
WHERE
    CAST(imdb_rating AS FLOAT) > 7.0
ORDER BY
    CAST(imdb_rating AS FLOAT) DESC
""").show(truncate=False)

```

6.1 Basic SQL Queries

Example: Basic SQL Query

```

# Run a SQL query
result = spark.sql("""
SELECT title, genre, imdb_rating
FROM disney_shows
WHERE imdb_rating > 7.0
""")

result.show(truncate=False)

```

This query selects shows with an IMDb rating greater than 7.0.

6.2 Aggregation and Grouping

Spark SQL supports standard SQL operations such as aggregation and grouping.

Example: Aggregating Data

```
# Aggregate and group data
genre_counts = spark.sql("""
SELECT genre, COUNT(*) as count
FROM disney_shows
GROUP BY genre
ORDER BY count DESC
""")

genre_counts.show(truncate=False)
```

```
genre_counts_not_null = spark.sql("""

SELECT COALESCE(genre, 'Unknown') AS genre, COUNT(*) as count
FROM disney_shows
GROUP BY COALESCE(genre, 'Unknown')
ORDER BY count DESC

""")

genre_counts_not_null.show()
```

```
genre_counts_not_null = spark.sql("""
SELECT genre, COUNT(*) as count
FROM disney_shows
WHERE genre IS NOT NULL
GROUP BY genre
ORDER BY count DESC
""")

genre_counts_not_null.show()
```

This query counts the number of shows in each genre and orders the results by count in descending order.

6.3 Handling Missing Data

You can handle missing data in SQL queries using functions like `COALESCE` and `IS NULL`.

Example: Handling Null Values

```
# Replace null values with a default value
cleaned_data = spark.sql("""
SELECT
    title,
    COALESCE(genre, 'Unknown') AS genre,
    COALESCE(imdb_rating, 0) AS imdb_rating
FROM disney_shows
""")

cleaned_data.show(truncate=False)
```

This query replaces null values in the `genre` and `imdb_rating` columns with 'Unknown' and 0, respectively.

6.4 Joining DataFrames

You can join multiple DataFrames using SQL.

Example: Self-Join

```
# Self-join to find shows with the same genre
genre_joins = spark.sql("""
SELECT a.title AS show1, b.title AS show2, a.genre
FROM disney_shows a
JOIN disney_shows b
ON a.genre = b.genre
WHERE a.title <> b.title
""")

genre_joins.show(truncate=False)
```

This query finds pairs of shows with the same genre.

6.5 Subqueries and Nested Queries

Spark SQL supports subqueries and nested queries, allowing for complex data retrieval.

Example: Subquery

```
# Find the shows with the highest IMDb rating in each genre
# Note: This subquery approach might not be the most efficient for large datasets
# A window function (as shown later) is generally preferred for this type of problem.
top_rated_shows_subquery = spark.sql("""
SELECT title, genre, imdb_rating
FROM disney_shows d1
WHERE imdb_rating = (
    SELECT MAX(imdb_rating)
    FROM disney_shows d2
    WHERE d1.genre = d2.genre
)
""")

top_rated_shows_subquery.show(truncate=False)
```

This query selects the show with the highest IMDb rating in each genre using a correlated subquery.

6.6 Working with Dates and Times

Spark SQL provides functions to handle date and time operations.

Example: Extract Year from Date

```
# First cast the released_at to date format, then extract year
shows_with_year = spark.sql("""
SELECT title,
       released_at,
       TO_DATE(released_at, 'dd MMM yyyy') AS released_date,
       YEAR(TO_DATE(released_at, 'dd MMM yyyy')) AS release_year
FROM disney_shows
""")

shows_with_year.show(5, truncate=False)
```

This query extracts the year from the `released_at` column.

Example: Filtering by Date Range

```
# Filter shows released after January 1, 2020
recent_shows = spark.sql("""
SELECT title, released_at
FROM disney_shows
WHERE TO_DATE(released_at, 'dd MMM yyyy') >= '2020-03-15'
""")
```

```
recent_shows.show(truncate=False)
```

This query selects shows released after March 15, 2020.

6.7 Advanced SQL Functions

Spark SQL includes advanced functions for data manipulation.

Example: String Functions

```
# Extract the first 10 characters of the plot
short_plots = spark.sql("""
SELECT title, SUBSTRING(plot, 1, 10) AS short_plot
FROM disney_shows
""")

short_plots.show(truncate=False)
```

This query extracts the first 10 characters from the `plot` column.

Example: Aggregating with Window Functions

Window functions perform calculations across a set of table rows that are related to the current row. They are powerful for ranking, moving averages, and cumulative sums.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

# Define the window specification:
# Partition by 'genre' (groups rows by genre)
# Order by 'imdb_rating' in descending order within each genre
window_spec = Window.partitionBy("genre").orderBy(col("imdb_rating").desc())

# Apply the window function using DataFrame API first
# This adds a 'rank' column based on the window specification
ranked_shows_df = disney_df.withColumn("rank", row_number().over(window_spec))

# Register the new DataFrame with the 'rank' column as a temporary view
ranked_shows_df.createOrReplaceTempView("ranked_shows")

# Now, query the temporary view using SQL to get the top-ranked show per genre
top_ranked_sql = spark.sql("""
SELECT title, genre, imdb_rating, rank
FROM ranked_shows
WHERE rank = 1
""")

top_ranked_sql.show(truncate=False)
```

This query ranks shows within each genre by IMDb rating and selects the top-ranked show per genre. It demonstrates how DataFrame API and Spark SQL can be combined.

7. Real Use Cases

- 1. Business Intelligence (BI):** Spark SQL is a backbone for BI dashboards and reporting. It can aggregate sales data, generate complex reports, and perform ad-hoc queries on large datasets to gain insights into business operations.
- 2. Data Warehousing:** In modern data warehousing solutions, Spark SQL is used for ETL (Extract, Transform, Load) operations, querying massive datasets stored in data lakes (e.g., Parquet, Delta Lake), and creating aggregated fact and dimension tables for analytics.
- 3. Data Cleaning and Transformation:** It provides a powerful and expressive way to clean, transform, and reshape raw data, handle missing values, and apply business rules to prepare data for analysis or machine learning models.

4. **Real-Time Analytics:** When integrated with Spark Streaming, Spark SQL can query and analyze live data streams (e.g., from Kafka, Kinesis), enabling real-time dashboards and anomaly detection.
5. **Ad-hoc Data Exploration:** Data scientists and analysts can quickly explore large datasets using familiar SQL syntax without needing to write complex programmatic code.

8. Conclusion

Spark SQL provides a powerful and flexible interface for querying and manipulating structured data at scale. By leveraging standard SQL queries within the Apache Spark ecosystem, you can perform complex data transformations, aggregations, and analyses efficiently. The deep integration of SQL with Spark's distributed processing capabilities, coupled with the intelligent Catalyst Optimizer, makes Spark SQL an indispensable tool for big data processing, suitable for both SQL-savvy users and developers building robust data pipelines. Understanding its core concepts, including the SparkSession, DataFrames/Views, the Catalog, and the Catalyst Optimizer, is crucial for anyone working with PySpark for data analysis and engineering.